

Bluetooth Sniffer

Privacy Analysis of Augmented Reality Glasses

1 Micro:bit

1.1 Overview

The micro:bit is a programmable microcontroller board. It features an array of sensors, a 5x5 LED display, and Bluetooth connectivity.

When flashed with a sniffer firmware, the Micro bit Bluetooth sniffer can be used to capture Bluetooth Low Energy (BLE) packets for basic BLE analysis. As of right now the btlejack firmware is the top choice. For our usage, it allows capturing BLE advertisements and following specific BLE connections using the access addresses.

1.2 Powering on

To simply power on the device to see if it works, one of the following can be chosen:

1. Plug the battery pack into the Micro bit. A red light should turn on.
2. Plug in the Micro bit to the computer using a USB cable. The red light plus another yellow light should turn on.

1.3 Set-Up for sniffing connections

The program that I tried to use and sniff BLE traffic is from <https://github.com/virtualabs/btlejack>

Btlejack allows sniffing, jamming and hijacking BLE (Bluetooth Low Energy) devices. It supports BLE versions 4.0 and limited 5.0, more information can be found in the link above.

The steps to set up the sniffer:

1. Check that Pip is installed. If it is not, then install it. The steps can be found in section 5.
2. Start a virtual environment, the steps can be found in section 6.
3. Now, install the 'btlejack' software with Pip:

```
sudo pip install btlejack
or
sudo pip3 install btlejack
```

4. Connect the Micro bit device to the computer with a USB cable.
5. Use the following command which will program every Micro bit device connected to the computer, preparing them for use with Btlejack. This is the step where we flash the sniffer firmware, btlejack, onto the Micro bit:

```
btlejack -i
```

6. There should be an output with:

```
Flashing ...
Flash n device
```

7. Scan existing connections and try to get the access address of active BLE devices with (try to make it so that the glasses and phone are in some kind of interaction):

```
btlejack -s
```

You should see something like this:

```
BtleJack version 2.1
```

```
[i] Enumerating existing connections ...  
[ - 88 dBm] 0x4aef4684 | pkts: 1  
[ - 87 dBm] 0x4aef4684 | pkts: 2
```

8. To target a specific connection, use step 7 and use the access address from the list of existing connections sniffed. The access address is a 32-bit that is assigned to each active BLE connection so that the devices are able to communicate with each other over a specific connection without interference from other nearby BLE connections.

```
btlejack -f 0x4aef4684
```

The following is what you should see. The number of sniffers detected is the number of Micro bits you are using:

```
BtleJack version 2.1  
  
[i] Detected sniffers:  
> Sniffer #0: fw version 2.1  
> Sniffer #1: fw version 2.1  
> Sniffer #2: fw version 2.1  
  
[i] Synchronizing with connection 0x4aef4684 ...  
✓ CRCInit = 0x555555  
✓ Channel Map = 0x1007ffffff  
✓ Hop interval = 8  
✓ Hop increment = 28  
[i] Synchronized, packet capture in progress ...
```

Note: If the connection you want to sniff is a BLE 5 connection, it needs to be specified that you want to target a BLE 5 connection, by including -5 in the command:

```
btlejack -f 0x4aef4684 -5
```

Note: If you want the capture to be saved as a PCAPNG file that you can open later with Wireshark for analysis, change the command to:

```
btlejack -f 0x4aef4684 -o name_of_file.pcapng
```

Note: that there is no exact identification that the connection sniffed is the glasses, but we can try to see whether the connection is coming from our device by:

- Turning off all nearby devices to reduce the number of active BLE devices to reduce the number of possible connections
- Using the RSSI value to get an idea about the proximity of the connection. RSSI (Received Signal Strength Indicator) gives a relative measure of the signal strength. This means that if your device (the glasses in our case) is close to the sniffer, it will have a stronger RSSI value, which is closer to zero (e.g., -40 dBm). So a way would be to place the glasses near the sniffer and observe the RSSI values of detected access addresses.
- Interacting with the glasses and see if the packet count increases with interaction.

1.4 Flashing a random program onto the Micro bit

Since nothing happened when I initially powered on the Micro bit, I tried to flash a random program to see if a program was actually being flashed onto the Micro bit.

1. Plug the Micro bit onto the computer.

2. One can write code either with MakeCode or on the micro:bit Python Editor.
3. Download the .hex file (your program) then drag and drop it onto the MICROBIT drive.
4. An example program that creates a heart can be found in the following link:
<https://microbit.org/get-started/user-guide/reset-your-microbit/>

1.5 Re-flash the Micro bit with the original firmware

When I encountered the errors on the Micro bit, I tried to re-flash the original firmware. The steps are also on the page in the following link, but to sum it up:

1. Download the .hex file from the link:
<https://microbit.org/get-started/user-guide/firmware/>
2. Press and hold on the reset button (same side as the lights, should be on the left side of the yellow light).
3. Plug in the Micro bit to the computer with a USB whilst pressing down on the button.
4. Afterwards, you should see MAINTENANCE, not MICROBIT, on your computer. Then you can let go of the button.
5. Check one of the three version of the Micro bit, this can be found on the bottom right corner (V1, V2.00, V2.20/21).
6. Download the correct version then drag and drop the .hex file onto the drive.
7. Next, wait for the yellow light to stop flashing and when the upgrade is completed, the Micro bit will reset.
8. Everything should be done one sees the MICROBIT drive appears on the computer.

1.6 Troubleshooting Errors

1.6.1 Flashing Micro bit

The following errors are what I encountered when trying to flash the Micro bit:

1. 504 with an unhappy face - indicates that there is an issue during the code upload or execution process. This can be caused if the code is corrupted or if there were issues during file transfer issues.
2. 528 with an unhappy face - indicates a runtime error related to flashing or executing code on the device. This can be caused if the firmware or .hex file flashed was corrupted or incompatible.

1.6.2 Permission issues regarding access

This permission issue is regarding the permission to access the serial port where the Micro bit is connected.

The error message looks like

```
...
PermissionError: [Errno 13] Permission denied: '/dev/ttyACMO'
...
```

One can solve this by giving the current user account permission to access the serial port. The steps are as follows:

1. Add the current user to the 'dialout' group:

```
sudo usermod -aG dialout 'USER'
```

To know who the current user, use the command:

```
whoami
```

2. Restart or log out and back in for the change to actually take effect.
3. One can verify if the change was made by check if the current username is in the 'dialout' group:

```
groups
```

One should then be able to continue running the bltejack commands.

1.7 Current issue

I was able to successfully flash the sniffer program, `blejack`, onto the microbit (see 1.3). However, the current issue is that the Micro bit is not able to sniff the connection between the phone and glasses. It is able to sniff some connections, but the RSSI value is too low for it to be from the glasses as they were placed right beside the sniffer also when I interact with the glasses, it does not affect the packet count. The Micro bit in general is not able to sniff much connections.

2 Adafruit

2.1 Overview

The Adafruit Bluefruit nRF51822 is a BLE (Bluetooth Low Energy) based on the Nordic Semiconductor nRF51822 chip.

When flashed with a sniffer firmware, it can be used to monitor BLE traffic between devices. For our usage, it allows capturing BLE packets and following specific BLE connections using the MAC address of the devices.

2.2 Powering on

To power on the device, simply plug in the Adafruit to your computer via USB. which a blue light should light up.

2.3 Set-Up for sniffing connections

For sniffing BLE traffic, I used the following Python-based sniffer program from Adafruit: https://github.com/adafruit/Adafruit_BLESniffer_Python

This python script allows for sniffing Bluetooth Low Energy devices. It supports BLE versions 4.0 and 5.0, more details and information can be found in the link above.

The steps to set up the sniffer:

1. Clone the git repository onto your computer

```
git clone https://github.com/adafruit/Adafruit_BLESniffer_Python.git
```

2. Go to the repository directory

```
cd Adafruit_BLESniffer_Python
```

3. Install the python dependencies necessary. This will require pip, so make sure that it is installed in your system, if not check section 4:

```
pip install -r requirements.txt
or
pip3 install -r requirements.txt
```

4. Connect the Adafruit device to the computer via USB.

5. Determine which serial port to use:

```
sudo dmesg | grep tty
```

This should return an output such as:

```
...: cp210x converter now attached to ttyUSB0
```

Another command that can be used is:

```
ls /dev/ttyUSB*
ls /dev/ttyACM*
```

The output returned should look something like this:

```
/dev/ttyUSB0
fish: No matches for wildcard 'ls/dev/ttyACM*'...
```

which means that exists a device path `/dev/ttyUSB0` for our connected device (the Adafruit) and there are no devices connected to your system with a path that matches `/dev/ttyACM*`.

6. Run the sniffer script (replace `'ttyUSB0'` with what you see in step 4):

```
python sniffer.py /dev/ttyUSB0
```

The script should output something like:

```
Capturing data to ~/Adafruit_BLESniffer_Python/Sniffer/capture.pcap
Connecting to sniffer on /dev/ttyUSB0
Scanning for BLE devices (5s) ...
Found 2 BLE devices:

[1] "" (49:C4:65:28:00:69, RSSI = -96)
[2] "" (61:A6:EF:82:97:DE, RSSI = -85)

Select a device to sniff, or '0' to scan again
>
```

7. To target a specific connection, use step 6 and compare the MAC address of your device to the list of devices, then use the respective number to sniff that particular connection. The MAC address of the glasses can be found in section 6. Type the corresponding number to focus the sniffer on that specific connection. For example:

```
Select a device to sniff, or '0' to scan again
> 2
```

All of the captures will be saved into the pcap file that is written on the top when you started the sniffer script. To stop the capture simply `Ctrl+C`.

2.4 Troubleshooting Errors

2.4.1 Adafruit not recognized

This is when computer is not able to find nor identify that Adafruit is plugged in. On my laptop, there was never a direct indication where I could see that Adafruit was plugged in. So to check if it is connected, I did the following:

1. Check to see if the device is recognized as a mass storage device:

```
lsblk
```

The Adafruit may not be recognized as a mass storage device (like the microbit) but rather connected as a serial device. One can see this by:

```
sudo dmesg | grep tty
```

The device can be recognized with something like the following, which means it is assigned to `(/dev/ttyUSB0)`:

```
...: cp210x converter now attached to ttyUSB0
```

2.4.2 Logger Path wrong

This seems to be an issue with the log file path in the `'Logger.py'` script in the python package. The error message looks like:

```
File ".../Adafruit_BLESniffer_Python/SnifferAPI/Logger.py", line 22, in <module>
logFilePath = os.path.join(
~~~~~
File "<frozen posixpath>", line 76, in join
TypeError: expected str, bytes or os.PathLike object, not NoneType
```

To solve this, change the configuration of the script. The steps are as follows:

1. Open the file 'Logger.py'

```
nano ../Adafruit_BLESniffer_Python/SnifferAPI/Logger.py
```

2. Go to the place where the 'logFilePath' is being defined, which should look something like this:

```
logFilePath = os.path.join(
    os.getenv('APPDATA'), 'Nordic Semiconductor', 'Sniffer'
)
```

3. Replace it with

```
logFilePath = os.path.join(
    os.getcwd(), 'Sniffer'
)
```

which should change it so that the log file will be saved in the current directory that is being worked on rather than some environment variable.

2.5 Current issue

I was able to successfully flash the sniffer program onto the Adafruit (see 2.3). However, the current issue is that the Adafruit is not able to sniff the connection between the phone and glasses. Although I usually am able to see a list of at least 20 different MAC addresses, the MAC address of the glasses does not appear in the list.

3 Ubertooth One

3.1 Overview

The Ubertooth One is a USB device able to scan, capture and monitor Bluetooth traffic across channels. It is built around the TI CC2591 RF front-end and the Nordic Semiconductor nRF51822 chipset which improves its range and signal strength for sniffing Bluetooth packets.

Unlike the Micro bit and Adafruit, it comes pre-flashed with firmware that allows it to function as a Bluetooth and BLE sniffer right out of the box. This means that you do not need to flash additional firmware. It is also able to sniff Classic Bluetooth, which none of the other devices supports. For our usage, it allows capturing both Classic Bluetooth and BLE packets and following specific connections using the MAC address of the devices.

3.2 Powering on

To power on the device, simply plug the Ubertooth one to your computer via USB, which 2 green lights and a red light should light up.

3.3 Set-Up for sniffing connections

Since it is already flashed, we do not need to flash a firmware onto the device. However, updates can be made to the firmware if new versions are released or if specific features or improvements are needed, check section 3.4 for more.

The steps to set up the sniffer:

1. Install the Ubertooth tools:

```
sudo apt install ubertooth
```

To test if the tools are correctly installed, try the following command to scan for Classic Bluetooth:

```
sudo ubertooth-scan
```

The output returned should look like:

Ubertooth API version 1.07 found, newer than that supported by libubertooth (1.06).
Things will still work, but you might want to update your host tools.

```
Ubertooth scan
systime=1730669467 ch=51 LAP=340d0e err=2 clk100ns=3760953 clk1=602
s=-82 n=-55 snr=-27
systime=1730669470 ch=53 LAP=48178b err=1 clk100ns=37301141 clk1=5968
s=-77 n=-55 snr=-22
```

```
Scan results:
??:??:?:34:0D:0E
AFH map: 0x00000000000008000000
??:??:?:48:17:8B
AFH map: 0x40040800000020000000
```

Compare the last three bytes of the MAC addresses in the scan result to that of your device. The MAC address of the glasses can be found in section 6. For me, the address of the glasses was not shown in the scan result. However, unlike the Micro bit and Adafruit where scanning a specific device relied on it being found when scanning existing connections in general, the Ubertooth one is able to target a device needing only the MAC address of the device. This is because it supports more sophisticated Bluetooth sniffing techniques.

The different commands to sniff for BLE can be found in 3.3.1 and the commands to capture Classic Bluetooth can be found in 3.3.2.

3.3.1 Sniff BLE

- Scans for BLE advertisements. The command captures BLE advertising packets without following connections nor capturing detailed data exchanged. Advertising packets include basic information like the device name, MAC address, and services advertised by BLE devices.

```
sudo ubertooth-btle -n
```

The output returned could look something like this:

```
systime=1730673973 freq=2402 addr=8e89bed6 delta_t=6.505 ms rssi=-88
40 19 04 4a c9 8d 17 56 02 01 1a 02 68 0c 0c ff 4c 00 10 07 27 9f 9d
20 8b 0d 18 47 f2 df
Advertising / AA 8e89bed6 (valid)/ 25 bytes
Channel Index: 37
Type: ADV_IND
AdvA: 56:17:8d:c9:4a:04 (random)
AdvData: 02 01 1a 02 68 0c 0c ff 4c 00 10 07 27 9f 9d 20 8b 0d 18
Type 01 (Flags)
00011010
LE General Discoverable Mode
Simultaneous LE and BR/EDR to Same Device Capable
(Controller)
Simultaneous LE and BR/EDR to Same Device Capable (Host)

Type 68
0c
Type ff (Manufacturer Specific Data)
Company: Apple, Inc.
Data: 10 07 27 9f 9d 20 8b 0d 18

Data: 04 4a c9 8d 17 56 02 01 1a 02 68 0c 0c ff 4c 00 10 07 27 9f
9d 20 8b 0d 18
CRC: 47 f2 df
```

- Sniffs nearby BLE. Listens for BLE traffic from active connections in range and sniffs packets exchanged.

```
sudo ubertooth-btle -p
```

The returned output should look like:

```
systime=1730673978 freq=2440 addr=92fa5232 delta_t=69.722 ms rssi=-92
09 00 a5 da b4
Data / AA 92fa5232 (valid) / 0 bytes
Channel Index: 17
LLID: 1 / LL Data PDU / empty or L2CAP continuation
NESN: 0 SN: 1 MD: 0

Data:
CRC: a5 da b4
```

- Sniffs BLE for a specific device. The -t option is used to follow active connections associated with the accompanying MAC address.

```
sudo ubertooth-btle -p -t98:59:49:34:7A:8C
```

- Sniffs BLE and saves the output to a file. The file format should be .pcapng, which can be later analyzed in Wireshark.

```
sudo ubertooth-btle -p -r name_of_file.pcapng
```

- Sniffs BLE for a specific device, determined by the MAC address, then saves it to a file that can be later analyzed using Wireshark.

```
sudo ubertooth-btle -p -t98:59:49:34:7A:8C -r name_of_file.pcapng
```

3.3.2 Sniff Classic Bluetooth

- Scans for Classic Bluetooth devices. It provides a list of nearby Classic Bluetooth devices by identifying their Lower Address Part.

The Lower Address Part (LAP) is a portion of the Bluetooth Address that is allocated by the device vendor. The LAP value uniquely identifies a Bluetooth device and is transmitted with each frame/packet as part of the packet header.

```
sudo ubertooth-scan
```

The expected output is already shown in 3.3.

- Captures nearby Classic Bluetooth. Listens for Classic Bluetooth traffic and captures packets from devices within range.

-rx is for raw captures. This is because we want to use Wireshark later, and other commands may not output data that is of compatible format to Wireshark (e.g. -dump, which outputs raw binary data for people who want to work with raw radio signals directly).

```
sudo ubertooth-rx
```

The output returned should look like:

```
systime=1730676995 ch= 0 LAP=48178b err=0 clkn=102 clk_offset=364 s=-72
n=-55 snr=-17
systime=1730676995 ch=17 LAP=48178b err=1 clkn=486 clk_offset=358 s=-77
n=-55 snr=-22
systime=1730676995 ch= 2 LAP=d6ecb9 err=0 clkn=689 clk_offset=4080 s=-75
n=-55 snr=-20
systime=1730676995 ch= 2 LAP=9645a8 err=1 clkn=763 clk_offset=5842 s=-72
n=-55 snr=-17
```

- Sniffs Classic Bluetooth for a specific device. The `-l` option is used to capture Classic Bluetooth traffic from a specific device identified by its LAP.
The LAP value is simply the last three bytes of the MAC address in hexadecimal form. For example, our glasses have the MAC address, 98:59:49:34:7A:8C. The last three bytes are 34:7A:8C, which means the LAP is 347A8C.

```
sudo ubertooth-rx -l 347A8C
```

- Captures Classic Bluetooth and saves the output to a file. The file format should be `.pcapng`, which can be later analyzed in Wireshark.

```
sudo ubertooth-rx -r name_of_file.pcapng
```

- Captures Classic Bluetooth for a specific device, determined by the LAP, then saves it to a file that can be later analyzed using Wireshark.

```
sudo ubertooth-rx -l 347A8C -r name_of_file.pcapng
```

3.4 Firmware version

To check the version of the firmware, the ubertooth tools need to be installed, if not check the steps to set up the sniffer below:

```
ubertooth-util -v
```

The output should show, which is as of now the latest version:

```
Firmware version: 2020-12-R1 (API:1.07)
```

3.5 Troubleshooting Errors

This occurred while I was trying to see how to update the firmware version. When trying to flash the firmware onto the device, it says that the firmware 'dfu' does not exist, then the directory for the firmware does not exist. The steps I tried to update the firmware:

1. Install the necessary tools for the update:

```
sudo apt-get install dfu-util
```

2. Download the latest firmware by cloning the Ubertooth repository

```
git clone https://github.com/greatscottgadgets/ubertooth.git
```

3. Go to that repository:

```
cd ubertooth/firmware
```

4. Flash the firmware in Device Firmware Update (DFU) mode of which I need to find the 'dfu' file. It was not in the firmware directory, so I tried to build it by:

```
sudo apt-get install cmake gcc-arm-none-eabi libnewlib-arm-none-eabi
cd ~/ubertooth/firmware
mkdir build
cd build
cmake ..
make
```

5. Where it then told me that the directory does not exist. I first checked if the Ubertooth repository exists, by:

```
ls ~/ubertooth
```

It did, so I assumed that the repository may be missing some files or something may not be set up correctly. So I installed a necessary library, `libbtbb`, which is for decoding and processing Bluetooth packets.

Either install it directly with:

```
sudo apt-get install libbtbb-dev
```

Or clone the repository and compile it manually:

```
git clone https://github.com/greatscottgadgets/libbtbb.git
cd libbtbb
mkdir build
cd build
cmake ..
make
sudo make install
```

After installation, refresh the library cache to make sure that the system can locate it:

```
sudo ldconfig
```

Installing it directly worked perfectly for me, but when I tried to compile it manually I had a warning related to Python package installation, that is likely due to a misconfigured Python environment. However, the warning shouldn't affect the primary functions of libbtbb needed for Ubertooth, so I am able to proceed with setting up Ubertooth itself.

Then go back to the Ubertooth repository and check that everything is set up:

```
cd ~/ubertooth/host
ls host
```

Check if there is a dfu file then we do not need to build from source. I did not have one so I continued with:

```
cd ~/ubertooth/host
mkdir build
cd build
cmake ..
make
sudo make install
```

Then it gave me the error that the build process failed due to an invalid version issue in the Python environment, which can happen when Python packages like setuptools or packaging interpret an empty version string as invalid. I tried to solve it by bypassing the python build like so:

```
sudo make -k install
```

-k allows 'make' to continue installing other components even if some Python-specific parts fail. Afterwards, the installation continued despite the errors, but some Python-related components were still causing some issues. However, since the core components like ubertooth-rx, ubertooth-tx, ubertooth-btle, and others were successfully built, I was able to continue sniffing for traffic and ignore the Python UI tools build fail. Although this did not solve the update, since it did not mess up the ability to sniff traffic and since I am already having the latest firmware, it did not cause more issues.

3.6 Current issue

I was able to sniff traffic, however, most if not all of the packets captured were Malformed. Also, the captures made were inconsistent for BLE as sometimes there was no source nor destination address provided.

4 nRF52840 Dongle

The nRF52840 Dongle is a USB device designed by Nordic Semiconductor. It is built on the nRF52840 chip that allows it to have a higher range for Bluetooth sniffing. The Dongle also needs to be flashed, and the process to set up the Dongle is more complex than the other devices. However,

when compared to other tools like the Ubertooth One, although the Dongle does not support Classic Bluetooth, it handles BLE more effectively due to its protocol support and better range. In the end, the nRF52840 Dongle was the most reliable and successful for BLE sniffing.

4.1 Set-Up for sniffing connections

For sniffing BLE traffic, I used the following Python-based sniffer program from Adafruit: https://github.com/adafruit/Adafruit_BLESniffer_Python

This python script allows for sniffing Bluetooth Low Energy devices. It supports BLE versions 4.0 and 5.0, more details and information can be found in the link above.

The steps to set up the sniffer:

1. Download nRF Connect for desktop from Nordic's nRF Connect website; <https://www.nordicsemi.com/Products/Development-tools/nrf-connect-for-desktop/download>
2. On Linux, the downloaded appimage needs to be made executable using the command:

```
chmod +x nrfconnect-5.1.0-x86_64.appimage
```

3. Plug in the nRF52840 USB Dongle via a USB port, which should result in some blinking light on the dongle.
4. Run nRF Connect for Desktop. On Linux, the command is:

```
./nrfconnect-5.1.0-x86_64.appimage
```

If you have the following error:

```
[33218:0108/194400.050889:FATAL:setuid_sandbox_host.cc(158)] The SUID sandbox helper binary
```

then use the command

```
./nrfconnect-5.1.0-x86_64.appimage --no-sandbox
```

5. In nRF Connect, install the Programmer application
6. Download the nRF52840 Dongle Hardware File on the following link: <https://www.nordicsemi.com/Products/Development-hardware/nRF52840-Dongle/Download>
7. Open the Programmer app in nRF Connect for Desktop.
8. On the top left, select the dongle connected to your computer.
9. Drag and drop the zip file downloaded into the Programmer app.
10. Click on Erase and write to flash the firmware.
11. Now the setup for the dongle is complete but in order to use it on Wireshark, some configurations need to be made on Wireshark.
12. Locate the extcap directory in Wireshark and copy the sniffer firmware zip file to the extcap directory.
13. Restart Wireshark and you should be able to see the nRF Sniffer as one of the interface under Capture Options.
14. Select the nRF Sniffer interface and begin scanning.

4.2

5 Install Pip

To check if Pip is installed write:

```
pip --version
or
pip3 --version
```

If it is not installed, first make sure that the system's package manager is up to date.

```
sudo apt update
sudo apt upgrade
```

Next, check if Python3 is installed. To check that write:

```
python3 --version
```

If Python3 is not installed, write:

```
sudo apt install python3
```

If it is already installed or once you have installed Python3, you can now install Pip using the command:

```
sudo apt install python3-pip
```

6 Set Up Virtual environment

The steps to set up a virtual environment:

1. One will need python3, so if it not installed write:

```
sudo apt install python3
```

2. One will also need the venv module, so if it is not installed write:

```
sudo apt install python3-venv
```

3. Create the virtual environment:

```
python3 -m venv myvenv
```

4. Start the virtual environment:

```
source myvenv/bin/activate
```

5. Then one should see '(myvenv)' in front of the terminal prompt.

6. To stop the virtual environment:

```
deactivate
```

7 Finding the MAC address of devices

7.1 Ray Ban Meta glasses

An application on the Android phone called 'nRF Connect' was used.

Once in the app, turn on the glasses.

After the light turns green on the notification LED on the glasses, refresh the 'scanner' page to find devices.

There should be a list of devices, check the 'Company' to see if the devices found are the glasses.

The 'Company' for the glasses should be 'Meta Platforms'.

The MAC address found for the glasses is 98:59:49:34:7A:8C.

7.2 RedMi Note 8 phone

In the settings app, go to 'About phone', 'All specs', 'Status'.

In there the MAC address found is b8:3b:cc:da:73:a8.